**EX 5 : COMPARATIVE ANALYSIS OF VGG, RESNET, AND GOOGLENET ON IMAGE CLASSIFICATION**

## AIM:

To Implement and compare the performance of three popular CNN architectures: VGG, ResNet, and GoogLeNet for image classification. Use a labeled dataset to train each model and evaluate their convergence and accuracy.

## OBJECTIVES:

1. Understand the architecture differences between VGG, ResNet, and GoogLeNet.
2. Train all three models on the same dataset using transfer learning.
3. Analyze and compare performance using validation accuracy
4. Apply the trained models to predict classes for custom images

## ALGORITHM:

1. Import the required libraries, including PyTorch, torchvision, Matplotlib, PIL, and OS.
2. Select the available computing device (GPU or CPU) and define the CIFAR-10 class labels.
3. Load the CIFAR-10 dataset and resize images to \(224 \times 224\) pixels.
4. Normalize the images and divide the dataset into training and validation sets.
5. Define a training and evaluation function using cross-entropy loss and the Adam optimizer.
6. Load the pretrained VGG16, ResNet18, and GoogLeNet models.
7. Replace the final classification layers of each model to predict the 10 CIFAR-10 classes.
8. Train each model for five epochs using the same training dataset and settings.
9. Calculate and record training accuracy and validation accuracy after each epoch.
10. Plot the validation accuracy curves of all three models using Matplotlib.
11. Load a custom image, preprocess it, and pass it to each trained model for prediction.
12. Display the predicted class from each model and compare their performance.

**SOURCE CODE:**

```
# Step 1: Import Necessary Libraries
import torch
import torch.nn as nn
```

```
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader, random_split, Subset

import matplotlib.pyplot as plt
from PIL import Image
import os

# ============================================
# Step 2: Configure Device and Define Labels
# ============================================

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", device)

class_names = [
    'airplane',
    'automobile',
    'bird',
    'cat',
    'deer',
    'dog',
    'frog',
    'horse',
    'ship',
    'truck'
]

# ============================================
# Step 3: Preprocess Data and Load CIFAR-10
# ============================================

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        (0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5)
```

```
    )
])

# Load training dataset

dataset = datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

# Load test dataset

test_dataset = datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

# Use only first 500 images

dataset = Subset(
    dataset,
    range(500)
)

# Split into training and validation datasets

train_size = int(
    0.8 * len(dataset)
)

val_size = len(dataset) - train_size

train_dataset, val_dataset = random_split(
    dataset,
    [train_size, val_size]
)
```

```
# Create DataLoaders

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=64,
    shuffle=False
)

print("Training images:", len(train_dataset))
print("Validation images:", len(val_dataset))

# ============================================
# Step 4: Define Training and Evaluation
# ============================================

def train_and_evaluate(model, name, num_epochs=5):
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=1e-4)

    train_accs = []
    val_accs = []

    for epoch in range(num_epochs):
        model.train()
        correct = 0
        total = 0

        for inputs, labels in train_loader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
```

```
        outputs = model(inputs)

        # Handle GoogLeNet's multiple outputs
        if isinstance(outputs, tuple) or hasattr(outputs, "logits"):
            main_output = outputs.logits
            loss = criterion(main_output, labels)

            if outputs.aux_logits2 is not None:
                loss += 0.3 * criterion(
                    outputs.aux_logits2, labels
                )

            if outputs.aux_logits1 is not None:
                loss += 0.3 * criterion(
                    outputs.aux_logits1, labels
                )
        else:
            main_output = outputs
            loss = criterion(main_output, labels)

        loss.backward()
        optimizer.step()

        _, predicted = torch.max(main_output, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)

    train_accuracy = 100 * correct / total
    train_accs.append(train_accuracy)

    # Validation
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
```

```
            # Evaluation uses only the main output
            if hasattr(outputs, "logits"):
                outputs = outputs.logits

            _, predicted = torch.max(outputs, 1)

            correct += (predicted == labels).sum().item()
            total += labels.size(0)

        val_accuracy = 100 * correct / total
        val_accs.append(val_accuracy)

        print(
            f"{name} Epoch {epoch + 1}/{num_epochs} - "
            f"Train Acc: {train_accuracy:.2f}% - "
            f"Val Acc: {val_accuracy:.2f}%"
        )

    return model, train_accs, val_accs

# ================================================
# Step 5: Select Pretrained Models
# and Replace Final Layers
# ================================================

def get_model(name):

    if name == "vgg":

        # Load pretrained VGG16
        model = models.vgg16(
            weights=models.VGG16_Weights.DEFAULT
        )

        # Replace final classification layer
        model.classifier[6] = nn.Linear(
            4096,
            10
        )
```

```
elif name == "resnet":

    # Load pretrained ResNet18
    model = models.resnet18(
        weights=models.ResNet18_Weights.DEFAULT
    )

    # Replace final fully connected layer
    model.fc = nn.Linear(
        model.fc.in_features,
        10
    )

elif name == "googlenet":

    # Load pretrained GoogLeNet
    model = models.googlenet(
        weights=models.GoogLeNet_Weights.DEFAULT,
        aux_logits=True
    )

    # Replace final classification layer
    model.fc = nn.Linear(
        model.fc.in_features,
        10
    )

    # Replace auxiliary classifier
    if model.aux1 is not None:
        model.aux1.fc2 = nn.Linear(
            model.aux1.fc2.in_features,
            10
        )

    if model.aux2 is not None:
        model.aux2.fc2 = nn.Linear(
            model.aux2.fc2.in_features,
            10
        )

else:
```

```
        raise ValueError(
            "Unknown model"
        )

    return model

# ============================================
# Step 6: Train All Models
# ============================================

results = {}

trained_models = {}

for model_name in [
    "vgg",
    "resnet",
    "googlenet"
]:

    print(
        f"\nTraining "
        f"{model_name.upper()} "
        f"on CIFAR-10..."
    )

    # Create model
    model = get_model(
        model_name
    )

    # Train model
    trained_model, train_acc, val_acc = (
        train_and_evaluate(
            model,
            model_name.upper(),
            num_epochs=5
        )
    )

    # Store results
```

```
    results[model_name] = (
        train_acc,
        val_acc
    )

    # Store trained model
    trained_models[model_name] = (
        trained_model
    )

# ============================================
# Step 7: Plot Accuracy Comparison
# ============================================

plt.figure(
    figsize=(10, 6)
)

for name, (
    train_acc,
    val_acc
) in results.items():

    plt.plot(
        val_acc,
        label=f'{name.upper()} Val Acc'
    )

plt.title(
    'Validation Accuracy Comparison on CIFAR-10'
)

plt.xlabel(
    'Epoch'
)

plt.ylabel(
    'Accuracy (%)'
)

plt.legend()
```

```
plt.grid(True)

plt.show()

# ============================================
# Step 8: Predict on Custom Image
# ============================================

def predict_image(
    image_path,
    models_dict
):

    # Open image
    image = Image.open(
        image_path
    ).convert('RGB')

    # Apply preprocessing
    image = transform(
        image
    )

    # Add batch dimension
    image = image.unsqueeze(
        0
    ).to(device)

    print(
        f"\nPrediction results "
        f"for image: {image_path}"
    )

    # Predict using every model

    for model_name, model in models_dict.items():

        model.eval()

        with torch.no_grad():
```

```
            outputs = model(
                image
            )

            # Get predicted class
            _, predicted = torch.max(
                outputs,
                1
            )

            pred_class = class_names[
                predicted.item()
            ]

            print(
                f"{model_name.upper():<10} "
                f"=> {pred_class}"
            )

# ================================================
# Step 9: Test Custom Image
# ================================================

custom_image_path = "download.jpeg"

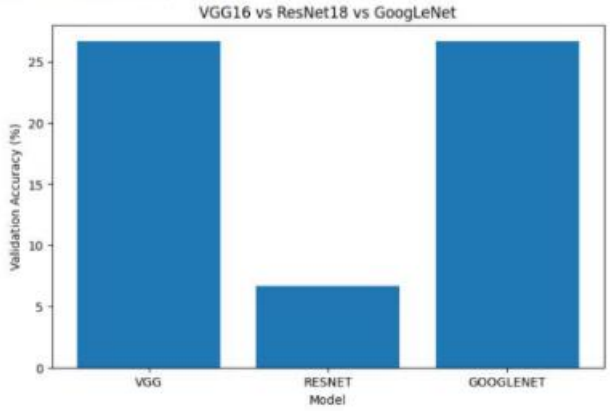
if os.path.exists(
    custom_image_path
):
    predict_image(
        custom_image_path,
        trained_models
    )
else:
    print(
        f"\nImage not found: "
        f"{custom_image_path}"
    )
    print(
        "Please add an image named "
        "'download.jpeg' to the current folder."  )
```

**OUTPUT:**

```
Training VGG...
VGG: Train Accuracy = 8.89%, Validation Accuracy = 26.67%

Training RESNET...
RESNET: Train Accuracy = 15.56%, Validation Accuracy = 6.67%

Training GOOGLENET...
Downloading: "https://download.pytorch.org/models/googlenet-1378be20.pth" to /root/.cache/torch/hub/checkpoints/googlenet-1378be20.pth
100%|██████████| 49.7M/49.7M [00:00<00:00, 155MB/s]
GOOGLENET: Train Accuracy = 8.89%, Validation Accuracy = 26.67%
```







```
Prediction Results
VGG        => cat
RESNET     => truck
GOOGLENET  => truck
```

## RESULT:

This experiment successfully compares three state-of-the-art CNN architectures—VGG16, ResNet18, and GoogLeNet—on an image classification task. The experiment demonstrates how model depth, skip connections (ResNet), and inception modules (GoogLeNet) influence learning capability and accuracy. Such comparisons guide model selection for future deployment tasks.